

Forge: A Certificate-Driven Proof Planner Beyond Lean’s grind

Abstract

A substantially more capable general-purpose Lean tactic should not replace `grind`’s effective local reasoning with a collection of disconnected solvers. It should discover the proof structure that makes local reasoning succeed: the right induction motive, an auxiliary lemma, an existential witness, a missing theorem instance, a nonlinear inequality certificate, or a necessary representation bridge. This article specifies *Forge*, a proposed certificate-driven proof planner that retains `grind` as a central engine and connects these capabilities through a typed, scope-aware graph of facts and obligations.

The proposal includes concrete search algorithms, certificate formats, Lean integration boundaries, a dependency and trust policy, worked mathematical examples, and an implementation roadmap. Five mechanisms are prototyped in Python: demand-directed Horn reasoning, structural induction with accumulator lemma synthesis, sparse rational nonnegativity certificates, Bernstein box certificates, and polynomial witness synthesis. The accompanying suite passes 69 tests; 13 persisted certificate bundles replay using only Python’s standard library. These results validate small algorithmic fragments, not a completed Lean tactic or a measured advantage over `grind`. The Lean reference scripts are supplied but were not compiled because a Lean runtime was unavailable.

Executive assessment

Recommendation. Implement Forge as a proof planner with a shared obligation graph, not as a new monolithic decision procedure. Its first major capability gain should be *automatic structural induction plus checked lemma discovery*. Its second should be *witness and theory-certificate search driven by the needs of the current proof*. Reuse existing Lean arithmetic, superposition, simplification, and sum-of-squares infrastructure wherever possible.

The critical distinction is between a proposed fact and a proved fact. Search engines may be heuristic, numerical, learned, or external. They may request additional assumptions, but they must not silently receive them. Only a proof of the exact requested statement, under explicitly recorded hypotheses, may enter the fact store. Completion means a closed proof of the original Lean goal with an approved axiom dependency set.

Status of this artifact. The architecture and Lean-facing interfaces are a design. The Python algorithms, tests, benchmark runner, saved certificates, and independent replay command are implemented and executed. No end-to-end `forge` tactic, Lean-kernel port of the Python checkers, or comparative Lean benchmark is claimed. The dated source audit uses Lean 4.34.0 and sources observed on September 14, 2026; mutable third-party sources are explicitly distinguished from that pinned baseline.

Contents

Executive assessment	1
1 The actual baseline: what must not be reinvented	4
1.1 A versioned comparison, not a caricature	4
1.2 Where a broader tactic can make a real difference	4
1.3 The meaning of “more powerful”	5
2 Existing libraries and the boundary of the contribution	5
2.1 Proof-search and external-prover components	5
2.2 Nonlinear arithmetic: reuse the existing SOS engine	6
2.3 Other components and non-goals	7
3 A precise architecture for Forge	7
3.1 The planner’s state	7
3.2 A small vocabulary of engine messages	8
3.3 Scope and dependency discipline	8
3.4 Why this is not a sequential tactic wrapper	9
4 Demand-directed quantified reasoning	9
4.1 Backward requests coupled to forward saturation	9
4.2 Variables absent from the conclusion	10
4.3 A measured branching example	10
4.4 Proof certificates and termination controls	10
5 Automatic induction and auxiliary-lemma discovery	11
5.1 Selecting an induction variable and motive	11
5.2 A fully worked accumulator example	11
5.3 The resulting proof, not just its tests	12
5.4 From failed residuals to stronger lemma synthesis	12
5.5 Preventing circular discoveries	13
6 Witness synthesis connected to proof obligations	13
6.1 A witness is a program with a proof obligation	13
6.2 An executed polynomial-witness example	13
6.3 Guards and dependent witnesses	14
7 Exact nonlinear reasoning and certificate search	14
7.1 The certificate language	14
7.2 The implemented sparse LP search	15
7.3 An executed quartic certificate	15
7.4 A deliberate incompleteness control	16
7.5 Production search: reuse, rational reconstruction, and escalation	16
7.6 Making nonlinear results useful to other engines	17
8 Exact Bernstein certificates on bounded domains	17
8.1 The representation and bound	17
8.2 A covering tree, not a collection of sampled boxes	17
8.3 An executed example and a termination qualification	18
9 Theory boundaries, SAT, and higher-order reasoning	18
9.1 Typed bridges rather than erased semantics	18
9.2 A conservative higher-order adapter	19
9.3 SAT and conflict learning	19
10 Trust profiles and final acceptance	19
10.1 Kernel checking and native evaluation are different profiles	19
10.2 An explicit acceptance procedure	20

10.3 Python certificates are not Lean proofs	20
11 Scheduling, retrieval, and proof-size control	21
11.1 A deterministic initial schedule	21
11.2 Library retrieval driven by missing premises	21
11.3 Avoiding repeated work and invalid caches	21
12 Concrete Lean implementation plan	22
12.1 Integration points that actually exist	22
12.2 Proposed module layout	22
12.3 A proposed user interface	23
12.4 Implementation stages with acceptance gates	23
13 Executed experiments and reproducibility	24
13.1 Environment and measured scope	24
13.2 Negative tests are part of the evidence	25
13.3 Reproduction commands	25
14 A fair end-to-end evaluation of “significantly more capable”	26
14.1 Why the environment and completion metric matter	26
14.2 Benchmark families and baselines	26
14.3 Metrics, ablations, and a concrete acceptance target	27
15 Limits, priorities, and conclusion	27
A Artifact map and status ledger	28
B Certificate schemas and their interpretation	28
C A compact replay target for the discovered lemma	29
Sources	30

1 The actual baseline: what must not be reinvented

1.1 A versioned comparison, not a caricature

The starting reference is the Lean language manual’s chapter on `grind`. It describes a proof-producing, SMT-inspired combination of congruence closure, propagation, quantified instantiation, case splitting, and theory reasoning. Its shared-state organization is already a major part of its strength. Adding an e-graph, arithmetic, or E-matching is therefore not, by itself, a meaningful advance over the baseline.¹

The release page resolved to Lean v4.34.0 during this audit, with a September 14, 2026 release date. The source-level baseline in this article is that tag, rather than an unspecified future meaning of `latest`. The executable was not available locally, so statements about its implementation are source observations, not newly run success or failure experiments.²

The pinned configuration enables more than the minimal description suggests: extensionality, function extensionality and function-valued congruence, associative-commutative reasoning, order reasoning, model-based theory combination, integer and linear arithmetic, and commutative-ring reasoning. There is also an optional library-suggestion path. Selected resource defaults are five E-matching rounds, generation eight, 1,000 instances, and nine splits. These are budget parameters, not semantic limits of Lean.³

The ring engine has tracked derivations for polynomial consequences, including superposition and simplification operations. Its proof objects retain algebraic side conditions associated with cancellation and characteristic. It is incorrect to describe it merely as a syntactic ring normalizer and then claim that adding Gröbner-style reasoning would be a new capability.⁴

The integer-arithmetic reference describes a linear fragment with inequalities, equalities, disequalities, and divisibility, including natural-number translation. Nonlinear expressions may be abstracted as atoms in the linear solver. A satisfying assignment to that abstraction is not necessarily a model of the original nonlinear formulas.⁵

1.2 Where a broader tactic can make a real difference

The opportunity is principally *proof organization*. A local solver may succeed once a human has chosen an induction, strengthened the induction hypothesis, supplied a useful identity, constructed a witness, selected the right theorem, or exposed the right representation. Forge should automate those choices and feed their checked consequences back into local reasoning.

This is a capability target, not a claim that every worked example below defeats current `grind`. Imported theorems, attributes, preprocessing, and tuning can change which tactic solves a particular goal. An example involving list reversal can become trivial when its correctness theorem is already available. A fair test must specify both the theorem statement and the environment in which it is presented.

¹Lean project, *The grind tactic*, language reference [1].

²Lean project, release v4.34.0 [2].

³Lean 4.34.0, `Init/Grind/Config.lean` [3].

⁴Lean 4.34.0, commutative-ring constraint and derivation types [4].

⁵Lean language reference, *Linear Integer Arithmetic* [5].

Capability	Baseline to retain	Forge’s proposed addition
Local equality and arithmetic	Existing grind theory engines	Request and reuse missing structural or nonlinear consequences.
Quantified reasoning	Existing E-matching and optional suggestions	Backward demand propagation, controlled new-term generation, and premise subgoals.
Recursive functions	Defining equations and local reasoning	Choose induction motives; generalize changing parameters; synthesize and prove auxiliary lemmas.
Existential conclusions	Any applicable existing logical rules	Search typed witness grammars and certify their universally quantified obligations.
Nonlinear inequalities	Existing arithmetic tactics and external SOS library	Reuse certified engines, coordinate guards, prioritize sparse certificates, and add box reasoning.
Trust and diagnostics	Lean proofs and solver traces	Exact-goal replay, explicit unresolved obligations, dependency audit, and reproducible proof artifacts.

1.3 The meaning of “more powerful”

Three different claims must be separated. *Broader coverage* means solving additional families of goals with comparable human input. *Better search* means solving more instances under a fixed resource budget. *Faster replay* means a smaller or cheaper proof after search has finished. One can improve coverage while worsening latency on easy goals.

Forge should expose a baseline-preserving configuration: run the original grind configuration before escalating, and retain its successful proof. With that original budget *plus additional resources*, this preserves its successes. Under an unchanged total wall-clock budget, no such dominance follows: orchestration overhead and budget allocation can lose cases. A non-starving unbounded schedule that eventually reruns every baseline attempt preserves the corresponding eventual successes, but that observation is not a practical performance theorem.

2 Existing libraries and the boundary of the contribution

2.1 Proof-search and external-prover components

Aesop already provides extensible proof search organized around goals and rules. Its AND/OR-style search and distinction between safer and branching rules are useful precedents and reusable infrastructure. Forge’s proposed distinction is the exchange of *typed demands and certified theory consequences* across that search, rather than the mere exis-

tence of backward search.⁶

Duper is a Lean-integrated proof-producing prover, and a natural candidate for logical and superposition reasoning. Its fact-selection interface matters: `duper [*]` requests local facts, whereas bare `duper` should not be assumed to consume them automatically. A Forge adapter should pass an explicit, recorded premise slice.⁷

Lean-auto provides a translation and automation interface between Lean and external reasoning. Its README distinguishes reconstruction-capable paths from standalone SMT or TPTP invocation modes without reconstruction. An external `unsat` result is therefore not a uniform proof interface. Forge must accept a reconstructed proof or create explicit remaining obligations; it must not promote a solver answer to a theorem.⁸

Lean-SMT reconstructs `cvc5` reasoning in Lean, but its documented interface can leave proof holes as Lean goals. That is useful partial progress, not completed verification. The adapter should import those goals as children of the current AND node and report success only after every child is discharged.⁹

QuerySMT is particularly close prior work: it uses SMT-generated hints together with Lean reasoning, including `grind` and `Duper`. Combining these systems is not a new proposal here. Forge’s additional design target is a persistent planner that can generate structural lemmas and witnesses, solve the resulting obligations, and let those results influence future search.¹⁰

2.2 Nonlinear arithmetic: reuse the existing SOS engine

A current `leanprover/sos` library already implements an end-to-end nonlinear-real-arithmetic tactic. Its documented pipeline includes reification, CSDP search, rational reconstruction, and certificate-based proof construction; it also provides a witness-oriented replay interface. Consequently, “add an SOS tactic to Lean” is not a novel component of Forge. The production design should reuse this library after compatibility and trust checks.¹¹

The inspected `SOS/Engine.lean` exposes `SOS.Engine.solve`, a structured problem/result interface, and deterministic result checking. Its configuration already includes degree deepening, constraint-product bounds, and optional refutation powers. Some README wording about fixed relaxation levels lags this source: the implementation is the stronger basis for planning integration. The separate `SOS.Core` module separates proof-facing data/checking from native search dependencies.¹²

There is a real compatibility task, not a ready-made common workspace. The retrieved SOS toolchain file specifies `v4.32.0-rc1`, whereas the inspected Lean baseline is `4.34.0`. That retrieved file may itself be a cached snapshot. Resolve commits and manifests together before attempting an integration; do not assume all projects’ moving main branches build with the same Lean release.¹³

For sparse polynomial data, `Hex.MvPoly` is a concrete candidate substrate. Its repository describes fixed-arity canonical sparse polynomials with an explicit monomial order and

⁶Aesop project documentation [6].

⁷Duper repository documentation [7].

⁸Lean-auto repository and interface paper [8, 9].

⁹Lean-SMT repository [10].

¹⁰Clune, Barbosa, and Avigad, *Hint-Based SMT Proof Reconstruction* [11].

¹¹Lean SOS project, README [12].

¹²Lean SOS source, `Engine.lean` and `Core.lean` [13, 14].

¹³Retrieved SOS `lean-toolchain` [15].

proved operation/evaluation laws. Forge should reuse a compatible proof-facing representation rather than repeatedly expand large expressions into unreduced Lean syntax.¹⁴

2.3 Other components and non-goals

Mathlib’s `linarith` infrastructure constructs proof terms from arithmetic certificates; `nlinarith` adds useful nonlinear preprocessing. These are first-line closers, not mechanisms to discard.¹⁵ Its `positivity` extensions cover compositional sign facts for arithmetic operations and other supported expressions, and should be tried before a costly numerical search.¹⁶

The `lean-egg` repository is now marked as no longer developed and directs users toward `grind`. It should not be the foundation of a new maintained Lean package. The Rust `egg` library remains a possible untrusted search component, but importing its data structures does not solve Lean’s binder, dependent-type, or proof-reconstruction problems.¹⁷

Induction and lemma synthesis are established research topics, not inventions of this article. Directed lemma synthesis for functional-program equivalence is a relevant source of stronger search heuristics. The present contribution is a concrete integration design and a small, inspectable executable study, not a claim to have originated these techniques.¹⁸

3 A precise architecture for Forge

3.1 The planner’s state

A search session has the conceptual state

$$S = (E, \Gamma, G, \mathcal{O}, \mathcal{F}, \mathcal{D}, \mathcal{C}, \mathcal{Q}, B).$$

Here E is an immutable environment snapshot; Γ is a typed local-context snapshot; G is the original target; $\mathcal{O}$ is an AND/OR obligation graph; $\mathcal{F}$ is a store of proved facts; $\mathcal{D}$ is a store of demands; $\mathcal{C}$ contains normalized terms and theory-specific representations; $\mathcal{Q}$ is a collection of work queues; and B is a multidimensional resource budget. “Immutable” refers to a snapshot identity: a branch can produce a successor snapshot, but another branch does not mutate it in place.

A fact record stores its proposition, proof, source scope, dependency support, and originating certificate identifier. A demand record stores a desired proposition or term shape, the consumer requesting it, the local context in which it is needed, and a bounded search plan. A demand carries *no logical authority*. Two consumers may share one demand only after their scopes and instantiations have been reconciled.

An obligation node is either a goal with alternative proof methods, an AND node requiring several subproofs, or a completed proof. A method such as applying a theorem creates an AND node for its premises. Choosing one of two theorems creates an OR choice.

¹⁴Hex multivariate polynomial library [16].

¹⁵Mathlib, `Mathlib.Tactic.Linarith.Frontend` [17].

¹⁶Mathlib positivity extensions [18].

¹⁷The `lean-egg` and `egg` repositories [19, 20].

¹⁸Sun et al., *Proving Functional Program Equivalence via Directed Lemma Synthesis* [21].

Induction creates an AND node for the recursor’s minor premises; an existential witness creates an AND node for its remaining properties.

3.2 A small vocabulary of engine messages

The following is an interface specification, not existing Lean API syntax:

```
EngineMessage :=
  Certified(proposition, proof, scope, dependencies)
| Need(proposition, scope, consumer)
| Refine(goal, subgoals, proofBuilder, savedContext)
| Candidate(certificateData, expectedStatement, obligations)
| Conflict(assumptions, checkedRefutation)
| Unknown(reason, resumableSearchState)
```

Candidate is intentionally separate from Certified. A numerical optimizer may return a polynomial decomposition, but its equality, sign conditions, interpretation map, and connection to the original Lean expression must be checked before a fact is emitted. An SMT engine may return a useful implication whose premises become demands. A failed external run emits Unknown, not a logical negation.

The proof builder in Refine is a typed Lean function or replay procedure that combines proofs of its exact subgoals into a proof of its parent. It is not an unchecked promise that some later tactic will make the relationship true. Before a method is admitted to the planner, its parent/child relationship must be elaborated in its saved context.

3.3 Scope and dependency discipline

Let a branch have assumptions $A_1, \dots, A_k$. If a solver proves P using A_2 and A_5 , the exported fact has support $\{2, 5\}$ together with all necessary local variables and their type dependencies. It may be reused in a descendant containing that support. It may be exported upward only as a corresponding implication, after the proof is abstracted over those assumptions. It is not valid to copy the bare proposition P into a sibling branch.

Computing support requires the dependency closure of free variables in both the proof and the types of those variables. This matters in Lean: a term $x : \text{Fin } n$ depends on n , and a local proof may mention a value only through a type. A syntactic set of visible hypothesis names is insufficient. Universe parameters and local typeclass instances are also part of the context.

Invariant 3.1 (Certified fact invariant). *Every fact visible at a node has a checked proof in that node’s local context, or a checked abstraction that is explicitly instantiated there. No candidate, relaxation model, sampled equation, or unresolved solver hint is a fact.*

Invariant 3.2 (Exact parent invariant). *Every completed obligation contains a proof of its recorded target in its recorded context. A goal may be normalized only together with a proof-producing transport back to the previous target.*

Proposition 3.3 (Compositional soundness of the planner design). *Assume the engine adapters produce valid Lean proofs or sound proof builders, and the two invariants are maintained. A completed root yields a Lean proof of G under Γ . Untrusted search choices cannot invalidate this conclusion.*

Proof. Proceed in a topological order through the completed proof DAG. A leaf is a checked fact or a checked direct proof. At an AND node, apply the checked proof builder to the already justified children. At an OR node, use the one completed alternative. Scope abstraction and instantiation preserve the stated contexts. The exact-parent invariant transports the final result to G . This is a specification-level argument: correctness of an eventual implementation still requires Lean checking of the assembled term and an audit of its dependencies. $\square$

3.4 Why this is not a sequential tactic wrapper

Consider a goal that requires a bound on a recursive accumulator. Induction produces a step obligation. A theorem instance suggests an inequality whose premise is a polynomial nonnegativity claim. The nonlinear engine proves that claim, which enables the theorem instance, which closes the induction step. The resulting inductive lemma then simplifies an existential-witness obligation. A list of tactics run once in sequence has no explicit representation of this feedback loop or of the fact that several consumers need the same intermediate claim.

Forge’s planner persists those obligations, schedules their producers, and reactivates waiting consumers when a proof arrives. A sophisticated existing tactic could implement such behavior internally; the point is to make the coordination protocol explicit and reusable across independent engines.

4 Demand-directed quantified reasoning

4.1 Backward requests coupled to forward saturation

Suppose the goal requires $P(t)$ and a theorem has the form

$$\forall \bar{x}, \quad H_1(\bar{x}) \rightarrow \dots \rightarrow H_m(\bar{x}) \rightarrow P(f(\bar{x})).$$

Match the conclusion against the requested atom, modulo already proved equalities. A successful substitution θ creates demands for $H_1\theta, \dots, H_m\theta$. When all are proved, instantiate the theorem and publish $P(f(\bar{x}))\theta$. This can construct intermediate terms that a purely ground-driven process has not yet produced.

The forward side remains essential. New proved equalities can enable previously blocked matches, and facts not directly anticipated by a goal can expose a contradiction. Use forward saturation as one queue and backward demands as another, with bounded widening rather than a permanent relevance filter. Requests are indexed by predicate/function head, type, and selected normalized argument features; theorem conclusions use a discrimination-tree index.

For multi-premise rules, order joins by estimated selectivity and share partial matches. Store which premise instances have already been tried. Newly arriving facts extend only joins that mention their indexes, in the spirit of semi-naïve evaluation. A failed join is not cached forever if it can be enabled by a later equality merge.

```
solveDemand(target, context, fuel):
  if a scoped proof of target exists: return it
  if fuel exhausted: return Unknown
  for theorem in conclusionIndex(target):
```

```

for substitution in boundedTypedMatches(theorem, target):
    premises := instantiatePremises(theorem, substitution)
    create shared child demands for premises
    when every child is proved:
        replay theorem application
        publish the checked target proof
schedule a fair forward-saturation slice

```

4.2 Variables absent from the conclusion

Backward matching does not determine every theorem parameter. In $\forall x y, R(x, y) \rightarrow P(x)$, a demand for $P(a)$ leaves y unknown. Forge must not silently choose an arbitrary inhabitant. It can enumerate appropriately typed terms from the local context, ask a witness engine, exploit equalities, or leave the application suspended with an existential-style obligation. Every selected instantiation is checked by Lean.

The executable Horn prototype deliberately handles only rules whose backward premises become ground after matching the head. It abstains on the example above even when a witness happens to exist. This sharply bounded fragment makes its behavior and certificate checker easier to audit; it is not the full quantifier algorithm proposed for Lean.

4.3 A measured branching example

Use an uninterpreted type, constant a , unary functions f, g , and rules

$$P(a), \quad P(x) \rightarrow P(f(x)), \quad P(x) \rightarrow P(g(x)).$$

The target is $P(f^d(a))$. The included forward baseline expands a breadth-first frontier and stops as soon as the target is produced. With the supplied rule order, it generates exactly 2^d distinct facts for $d \geq 1$: the preceding levels contain $2^d - 1$ facts, and the first target at the next level adds one. A backward proof uses $d + 1$ fact nodes.

This is a separation of *these two implemented Horn search policies*. It is not a measurement of grind, whose goal internalization, congruence classes, triggering, heuristics, and resource policy are not modeled by this baseline. The experiment demonstrates why demand information can matter; it does not establish a Lean speedup.

The logical proof has linear size in d , but the Python implementation uses nested term objects. Hashing and comparing such terms can add depth-dependent costs. A production hash-consed term DAG is needed before translating the node count into an approximately linear implementation cost.

4.4 Proof certificates and termination controls

A Horn certificate is a topologically ordered DAG. Each node is either an explicit premise or a named rule instance whose parent indices are smaller than the node index. The checker independently matches the rule's head and all premises under one consistent substitution. The final node must be the exact requested goal.

Cycle detection prevents immediate recursive demand loops. It is not a completeness argument for arbitrary recursive Horn programs: an active request may have another finite derivation that a particular bounded policy misses. The planner should use iterative

depth bounds, table successful answers, and retain resumable alternatives. A timeout or active-cycle rejection must be reported as absence of a certificate, not as falsity.

5 Automatic induction and auxiliary-lemma discovery

5.1 Selecting an induction variable and motive

Induction should be an explicit planner action, not an unconditional tactic applied to every natural number or list. Analyze unresolved recursive applications after cheap simplification. For each application, inspect the defining equations and identify structurally decreasing arguments, parameters that change in recursive calls, and dependencies of the target on those arguments.

Generate a bounded collection of plans: ordinary structural induction on a visible datatype variable; functional induction using the definition’s available induction principle; and, later, well-founded induction with a supplied or proved decrease relation. Initially prefer one major premise. Trying arbitrary nested inductions before examining the residual goal is a recipe for a large search tree.

A useful ranking function rewards recursive occurrences that become constructor applications and penalizes the number of constructors, dependent premises, and new generalized variables. Ranking affects only search order. The actual induction step must be performed by applying a Lean recursor or a checked induction theorem; Forge must not manufacture an unrestricted “induction hypothesis” as an ordinary axiom.

The motive is the main source of additional capability. If a recursive call changes an accumulator, an induction hypothesis about one fixed accumulator may be useless. Generalize that parameter before induction. In a dependent context, also revert hypotheses whose types depend on a generalized variable, in a valid dependency order, and reintroduce them in each minor premise. The target then quantifies over the appropriate parameters and their assumptions.

5.2 A fully worked accumulator example

Write list concatenation as `app`. Define

$$\begin{aligned} \text{app}([], a) &= a, & \text{app}(x :: xs, a) &= x :: \text{app}(xs, a), \\ \text{rev}([]) &= [], & \text{rev}(x :: xs) &= \text{app}(\text{rev}(xs), [x]), \\ \text{revAcc}([], a) &= a, & \text{revAcc}(x :: xs, a) &= \text{revAcc}(xs, x :: a). \end{aligned}$$

The original target is

$$\forall xs, \quad \text{revAcc}(xs, []) = \text{rev}(xs). \quad (1)$$

A direct induction yields an induction hypothesis about $\text{revAcc}(xs, [])$, but the step contains $\text{revAcc}(xs, [x])$. The desired strengthening is

$$\forall xs a, \quad \text{revAcc}(xs, a) = \text{app}(\text{rev}(xs), a). \quad (2)$$

The implemented prototype does not simply return the right-hand side of (2). It notices that the second argument of the recursive `revAcc` call changes, replaces the original fixed accumulator by a variable, and enumerates a small right-hand-side grammar:

$$e ::= xs \mid a \mid [] \mid \text{rev}(e) \mid \text{app}(e, e).$$

Terms are enumerated in increasing syntax-tree size. There are 49 concrete sample environments: every pair of lists of length at most two over a two-element alphabet. Samples only reject candidates. The first surviving candidate that receives an induction certificate is the 41st enumerated term, $\text{app}(\text{rev}(xs), a)$.

The actual generalization recognizer is deliberately specific to the revAcc equation. The general motive-analysis algorithm above is a design for the Lean implementation, not an already implemented generic recursion analyzer. This distinction prevents a tiny successful example from being misrepresented as general automatic induction.

5.3 The resulting proof, not just its tests

First prove concatenation's right identity and associativity by induction. These are named, separately checked lemmas in an acyclic theorem bank. The base case of (2) is $\text{revAcc}([], a) = a = \text{app}([], a)$. For the step, assume

$$\forall a, \quad \text{revAcc}(xs, a) = \text{app}(\text{rev}(xs), a).$$

Then

$$\begin{aligned} \text{revAcc}(x :: xs, a) &= \text{revAcc}(xs, x :: a) \\ &= \text{app}(\text{rev}(xs), x :: a) \\ &= \text{app}(\text{app}(\text{rev}(xs), [x]), a) \\ &= \text{app}(\text{rev}(x :: xs), a). \end{aligned}$$

The middle equality uses associativity and the defining equation for appending a singleton. Specializing the proved lemma at $a = []$ and using right identity proves (1).

The prototype independently checks the base and step rewrite traces. In the step, the tail is a *rigid eigenconstant*; only explicitly generalized parameters remain instantiable in the induction hypothesis. In the base case, the induction hypothesis is not available at all. These restrictions are essential. Treating the tail as an ordinary rewrite metavariable would allow an induction hypothesis to rewrite the original goal at an unjustified larger argument.

5.4 From failed residuals to stronger lemma synthesis

A production synthesizer should use failure information instead of enumerating all small theorems. Normalize both sides of an unclosed induction step and identify the unmatched recursive skeletons. Compute typed anti-unifiers of the original target and those residuals, recording substitutions back to each. Candidate lemmas should relate the same operations while replacing obstructing subterms by fresh parameters.

There are three useful candidate sources. First, generalize a changing non-structural argument, as above. Second, propose a homomorphism/distribution lemma such as $\text{rev}(\text{app}(xs, ys)) = \text{app}(\text{rev}(ys), \text{rev}(xs))$ when nested calls obstruct normalization. Third, synthesize a stronger invariant containing a measure, bound, or relation required by the step. Each candidate becomes its own proof obligation, never a rewrite rule merely because it helped simplify another candidate.

Use a typed, bounded grammar assembled from operations present in the residual, relevant constructors, and a small retrieved lemma vocabulary. Eliminate candidates that fail executable tests when testing is available. Quotient syntactically different candidates

only by *already proved* equalities; otherwise the quotient can discard the correct expression or conceal an unproved assumption. After a failed universal proof, retain the candidate as unproved; a failure need not yield a counterexample.

A stronger future implementation can replace broad enumeration with the induction-friendly transformations studied in directed lemma synthesis. That is an algorithmic development path, not a claim that the included enumerator implements that research system.

5.5 Preventing circular discoveries

Each candidate lemma has a dependency set. It may use previous accepted lemmas and its own legally scoped induction hypotheses. It may not use itself as a global theorem, the original unproved target, or another candidate that depends back on it. The simplest initial policy is an append-only theorem bank ordered by accepted proofs. More elaborate mutually inductive schemes require an explicit joint induction principle, not a cycle in a generic dependency graph.

The prototype’s bank checks five list lemmas: concatenation right identity, concatenation associativity, the discovered accumulator specification, reversal over concatenation, and reversal involutivity. Only the accumulator specification’s right-hand side is synthesized. The other four are supplied statements whose induction certificates are produced and replayed. All are about a small freely generated list fragment, not arbitrary dependent Lean objects.

6 Witness synthesis connected to proof obligations

6.1 A witness is a program with a proof obligation

For a goal $\exists y : \tau, P(y)$, create a typed grammar for y from the local context and relevant constructors. The alternatives are not just constant terms: after universal variables have been introduced, a witness may depend on them. Instantiating a candidate w produces the exact child goal $P(w)$. Success requires a proof of that child, followed by the ordinary existential introduction rule.

For a goal $\forall x, \exists y, P(x, y)$, sampling individual values of x and finding unrelated values of y does not construct a function or prove the theorem. The grammar must represent a well-typed expression $w(x)$, or the planner must produce another legitimate Lean proof of the existential statement. A synthesis oracle may help choose w but cannot justify $P(x, w(x))$ by testing.

6.2 An executed polynomial-witness example

Consider

$$\forall x \in \mathbb{R}, \quad \exists y \in \mathbb{R}, \quad x \leq y \wedge -x \leq y.$$

The prototype enumerates $w(x) = a + bx + cx^2$ with $a, b, c \in \{-2, -1, 0, 1, 2\}$, ordered first by coefficient ℓ_1 size and then lexicographically. Five values $x = -2, -1, 0, 1, 2$ are used only as a rejection filter. The 23rd candidate is $w(x) = x^2 + 1$; three candidates reach universal certificate search before acceptance.

The certificate engine separately proves $w - x \geq 0$ and $w + x \geq 0$ over all real inputs. A compact mathematical presentation is

$$x^2 + 1 - x = (x - \frac{1}{2})^2 + \frac{3}{4}, \quad x^2 + 1 + x = (x + \frac{1}{2})^2 + \frac{3}{4}.$$

These are convenient explanatory decompositions. The numerical LP run actually returned different rational weighted-square decompositions, retained verbatim as data in the certificate file; both replay exactly. The artifact does not pretend that a hand-simplified certificate is the optimizer’s original output.

The example illustrates cross-component behavior: witness enumeration requests two universal nonlinear facts, and a theory certificate closes them. A production implementation should also try simpler witnesses such as an available absolute-value or maximum expression when the local library makes those cheap. Restricting the prototype to polynomials is an experiment design, not a recommended restriction on all Forge witnesses.

6.3 Guards and dependent witnesses

A candidate involving division, an array lookup, or a dependent constructor may require additional evidence. That evidence becomes an explicit child goal. For example, constructing a value of `Fin n` requires a value and a bound; constructing a term with a nonzero-denominator simplification requires the appropriate nonzeroness statement. Shared guard demands should be solved once and reused with their proofs.

The planner should distinguish candidate elaboration failure from mathematical refutation. It should also preserve dependencies between witness metavariables: a later witness may depend on an earlier one. Eagerly closing a goal using a metavariable that another branch later instantiates is unsafe as a search-state operation even when Lean would ultimately reject the resulting term.

7 Exact nonlinear reasoning and certificate search

7.1 The certificate language

Normalize a polynomial target over an explicitly selected ordered field. Let $g_j \geq 0$ be proved inequality guards and $h_k = 0$ proved equality guards. A general certificate can have the form

$$p = \sum_J \sum_i c_{J,i} q_{J,i}^2 \prod_{j \in J} g_j + \sum_k u_k h_k, \quad c_{J,i} \in \mathbb{Q}_{\geq 0}. \quad (3)$$

The finite index J can be a set or a multiset, depending on the search policy. The polynomials $q_{J,i}$ and u_k have rational coefficients. The equality is checked exactly, not by floating-point residuals.

Proposition 7.1 (Conditional cone certificate). *If (3) holds as a polynomial identity and the stated guards hold at an input, then $p \geq 0$ at that input.*

Proof. Each square is nonnegative, each coefficient is nonnegative, and each guard product is nonnegative. Their sum is nonnegative. Every equality-ideal term $u_k h_k$ evaluates to zero. Evaluating the polynomial identity gives the claim. $\square$

This simple argument is the core reason an unreliable optimizer can be useful. It searches for data; a small checker verifies a mathematical identity and a finite list of sign conditions. The typed interpretation from rational polynomials to the original Lean field is part of the proof, not an implicit casting convention.

The Python certificate checker implements the nonnegative-cone part, without equality-ideal multipliers. It accepts arbitrary sparse polynomials as square arguments, but its search grammar is much more restricted. A successful check is conditional on the supplied guards; the checker does not prove those guards from a Lean context.

7.2 The implemented sparse LP search

Let $D = \deg p$. For each guard product g_J of bounded length, enumerate monomials of degree at most

$$d_J = \left\lfloor \frac{D - \deg g_J}{2} \right\rfloor.$$

Create a dictionary containing each monomial m and each binomial $m + cn$ for distinct monomials and $c \in \{-2, -1, -\frac{1}{2}, \frac{1}{2}, 1, 2\}$. The corresponding candidate columns are $q^2 g_J$. Flatten their rational coefficients into a linear system

$$Aw = b, \quad w \geq 0.$$

The prototype asks SciPy's HiGHS LP interface to minimize $\sum_i w_i$ subject to these constraints.¹⁹

The returned floating-point weights are rationalized with a denominator cap of 10^6 . Tiny weights may be dropped during *search reconstruction*; this never weakens the final check. The reconstructed identity must match the target exactly using `fractions.Fraction`. A residual of 10^{-12} is not a proof. If exact replay fails, the result is Unknown even when the numerical solver reports success.

With $M = \binom{n+d}{d}$ monomials, the unguarded dictionary has at most $M + 6\binom{M}{2}$ entries before deduplication. Guard products increase this count combinatorially. Sparse coefficient storage, incremental degree bounds, coefficient-size limits, and proof-size limits are therefore necessary. The LP prototype is intentionally small; it is not a replacement for the existing Lean SOS library's more general SDP machinery.

7.3 An executed quartic certificate

For

$$p = x^4 + y^4 + z^4 - x^2 y^2 - y^2 z^2 - z^2 x^2,$$

the search constructs 280 candidate columns and 35 coefficient equations. It finds a three-term certificate:

$$p = \frac{1}{2}(x^2 - y^2)^2 + \frac{1}{2}(y^2 - z^2)^2 + \frac{1}{2}(z^2 - x^2)^2. \quad (4)$$

The saved certificate is checked by exact coefficient comparison. This is a useful compact replay target, but it is not evidence that other existing Lean tactics cannot solve the theorem. In fact, supplying the three square facts makes the final arithmetic proof straightforward; the automation question is how to find and use those facts without human intervention.

¹⁹SciPy documentation, `linprog(method="highs")` [22].

Other executed examples include $xy \geq 0$ from $x, y \geq 0$, $x^3 \geq 0$ from $x \geq 0$, $x - x^2 \geq 0$ from $x, 1 - x \geq 0$, and

$$x^3 + y^3 - xy(x + y) = (x + y)(x - y)^2 \geq 0 \quad \text{when } x, y \geq 0.$$

These exercise conditional guard products and squares, not just unguarded polynomial normalization.

7.4 A deliberate incompleteness control

The true polynomial $(x + y + 1)^2$ has an obvious one-square certificate, which the checker accepts when explicitly supplied. The restricted LP dictionary fails to find a certificate for it. This failure is expected: a rank-one quadratic Gram matrix for the coefficient vector $(1, 1, 1)$ cannot be decomposed as a positive sum of rank-one vectors with only two nonzero coordinates, unless the summands lie in its one-dimensional range; none of the allowed nonzero binomial vectors do.

Thus the *checker* supports a larger certificate language than this *searcher* can discover. The experiment is a useful protection against an inflated “SOS-complete” claim. A production solver should escalate to general Gram-matrix search or use the existing Lean SOS engine, rather than interpret this failure as a negative mathematical result.

7.5 Production search: reuse, rational reconstruction, and escalation

The recommended nonlinear adapter first tries cheap sign reasoning and existing arithmetic closers. It then tries a sparse dictionary search for a compact certificate, followed by the compatible Lean SOS engine. Only afterward should it increase degree, constraint-product, or refutation budgets. Those escalation parameters already exist in the inspected SOS engine; Forge’s contribution is to allocate them according to outstanding proof obligations and reuse their results.

For a general SOS block, write $\sigma = v^T Q v$ in a monomial basis v , search for a positive semidefinite Gram matrix Q , and recover rational data. A weighted-square certificate can be extracted from an exact rational factorization $Q = LDL^T$ with nonnegative diagonal D . Zero pivots require care: a zero pivot is admissible only when the relevant residual off-diagonal entries are zero, or after a justified permutation/reconstruction. A floating-point “nearly positive semidefinite” matrix is not adequate.

CSDP is a concrete open-source SDP search backend; the existing Lean SOS integration is preferable to building a second unaudited wrapper.²⁰ A fallback oracle such as SymPy can suggest exact polynomial identities or perform differential tests, but a returned identity must still be checked in the chosen certificate representation.²¹

To prove strict positivity, search for $p - \varepsilon \geq 0$ with an exact $\varepsilon > 0$, or use a properly justified strict-constraint refutation. A fixed global margin is sufficient but not necessary: $x > 0$ implies $x > 0$ without a uniform rational lower bound. For infeasibility, a certificate for -1 in a nonnegative cone plus the equality ideal gives a contradiction. If $q > 0$ is a strict hypothesis, a checked identity expressing $-q^m$ as a nonnegative cone plus vanishing ideal terms, with $m \geq 1$, also gives a contradiction. All strictness and exponent side conditions must be retained.

²⁰The COIN-OR CSDP project [23].

²¹SymPy polynomial tools reference [24].

7.6 Making nonlinear results useful to other engines

A successful certificate should not be used only to close the current goal. It can publish an inequality needed by a theorem application, a branch guard, a witness obligation, or an induction invariant. Conversely, a linear solver can publish proved bounds that tighten nonlinear search. The planner can rank candidate nonlinear consequences by the number of waiting consumers and the estimated cost of their certificates.

One concrete extension is to introduce an abstraction variable $z = xy$ together with checked box bounds $\ell_x \leq x \leq u_x$ and $\ell_y \leq y \leq u_y$. The four elementary product inequalities yield

$$\begin{aligned} z &\geq \ell_x y + \ell_y x - \ell_x \ell_y, \\ z &\geq u_x y + u_y x - u_x u_y, \\ z &\leq u_x y + \ell_y x - u_x \ell_y, \\ z &\leq \ell_x y + u_y x - \ell_x u_y. \end{aligned}$$

For example, the first follows from $(x - \ell_x)(y - \ell_y) \geq 0$. These are proved linear consequences once the box assumptions and $z = xy$ are established. A linear model violating $z = xy$ can guide refinement, but it is not a counterexample to the original goal. This bound/cut exchange is proposed; it is not implemented in the supplied prototype.

8 Exact Bernstein certificates on bounded domains

8.1 The representation and bound

For a box $B = \prod_i [a_i, b_i]$ with rational endpoints and $a_i < b_i$, substitute $x_i = a_i + (b_i - a_i)t_i$. Express the resulting polynomial on $[0, 1]^n$ as

$$p = \sum_{0 \leq k_i \leq d_i} \beta_k \prod_{i=1}^n \binom{d_i}{k_i} t_i^{k_i} (1 - t_i)^{d_i - k_i}.$$

Every basis function is nonnegative and the basis functions sum to one. Consequently, $\min_k \beta_k \leq p \leq \max_k \beta_k$ on the box.

If the power-basis coefficients after substitution are a_j , the implemented conversion is

$$\beta_k = \sum_{j \leq k} a_j \prod_i \frac{\binom{k_i}{j_i}}{\binom{d_i}{j_i}}.$$

All arithmetic uses exact rationals. A leaf certificate stores a claimed lower bound, but the checker recomputes it; the claim itself is never trusted.

8.2 A covering tree, not a collection of sampled boxes

When a leaf bound is insufficient, bisect its longest side at a rational midpoint. An internal certificate node records the split axis and split value and has exactly two children. The checker reconstructs the two child boxes from the parent, verifies that the split is interior, and checks both children. This inductive construction proves coverage automatically. Accepting a list of independently positive boxes without checking that they cover the original domain would leave a soundness gap.

Proposition 8.1 (Box-certificate soundness). *If every leaf of a checked covering tree has a nonnegative Bernstein lower bound, then $p \geq 0$ throughout the original box. If every leaf bound is strictly positive, then $p > 0$ there.*

Proof. The Bernstein convex-combination identity proves the leaf claims. Each internal node’s children cover its box, including their common boundary. Induction over the tree transports the property to the root. $\square$

8.3 An executed example and a termination qualification

For $p(x) = x^2 - x + 3/10$ on $[0, 1]$, the initial Bernstein coefficients are

$$(3/10, -1/5, 3/10).$$

Their negative minimum is inconclusive, not a refutation. Splitting at $1/2$ produces two leaves, each with exact lower bound $1/20 > 0$. The certificate has three nodes and proves strict positivity on the requested interval.

The polynomial is actually positive on all of $\mathbb{R}$; an SOS proof is simpler. The bounded-domain experiment is intentionally used to exercise a different certificate mechanism. It should not be advertised as a case that demands Bernstein subdivision or defeats the SOS engine.

For a polynomial with a positive minimum on a compact box, sufficiently fine uniform subdivisions yield positive Bernstein lower bounds. To see the mechanism, expand around a subbox endpoint. Under affine rescaling, every nonconstant coefficient contains at least one side-length factor; the correction of a Bernstein coefficient from the endpoint value tends uniformly to zero as all side lengths tend to zero. A positive minimum eventually absorbs that correction. This explains eventual success with unbounded fair subdivision, not any useful a priori complexity guarantee.

If the minimum is zero, the same argument gives no finite strict certificate; a true non-strict inequality can also remain hard for a chosen subdivision policy. Production Forge should try symbolic factorization, exact boundary analysis, or another certificate language before subdividing indefinitely. The tensor-product basis has $\prod_i (d_i + 1)$ coefficients, so dimension is an important practical limit.

9 Theory boundaries, SAT, and higher-order reasoning

9.1 Typed bridges rather than erased semantics

Sharing proved propositions is safer than assuming every solver’s internal objects have interchangeable meanings. Each reifier should return a typed interpretation, an atom environment, and a proof that the encoded target denotes the original expression. If two solvers use different abstractions of a term, exchange a checked bridge theorem or retain the representations separately.

For example, rational polynomial algebra treats subtraction as additive inversion. Natural-number subtraction is truncated. Translating $(a - b : \text{Nat})$ to a field difference requires a guard such as $b \leq a$ and an appropriate transport theorem; otherwise one must keep the operation opaque or split the relevant cases. Integer division is not field division, and fixed-width addition is not unbounded integer addition. These distinctions should be visible in the engine capability descriptors.

Unknown real-valued applications can be polynomial atoms without being uninterpreted everywhere. One engine may know a theorem about $f(x)$ while the polynomial engine treats its value as one coordinate. Once that theorem gives a bound or equality, the bound is proved in the original context and can be translated into the polynomial representation. No global erasure of dependent types is necessary.

9.2 A conservative higher-order adapter

Extensionality is already present in the baseline. Forge’s higher-order plan is to choose useful extensionality steps, expose resulting pointwise demands, and use a reconstruction-capable Duper/Lean-auto route when appropriate. Bound-variable identity, universe levels, and typeclass instances must remain part of theorem matching. Matching under binders should use Lean’s own typed expression machinery, not the Python prototype’s first-order matcher.

Monomorphization and translation to first-order or higher-order external languages need explicit fragment checks and proof-producing transport. Empty types, dependent function domains, and quantifier well-formedness must not be silently replaced by the external solver’s assumed nonempty sorts. Unsupported constructs should remain opaque where a sound abstraction permits that, or cause the adapter to abstain.

9.3 SAT and conflict learning

For Boolean or fixed-width bitvector obligations, a SAT-oriented backend may be much more suitable than broad quantified saturation. Forge can also use a checked conflict clause as a learned fact. If a branch refutation depends on assumptions $A_{i_1}, \dots, A_{i_r}$, derive the corresponding clause $\neg A_{i_1} \vee \dots \vee \neg A_{i_r}$, with its proof. Conflict analysis may suggest a smaller support set, but replay must justify the actual clause.

A proposed future CDCL-style coordinator can use these clauses to avoid repeated branches. The first implementation should reuse existing solver reasoning and branch-state machinery instead of writing another full SAT/SMT solver. The included prototype has no CDCL implementation, bit-blaster, or higher-order prover.

Do not assume a generic Nelson–Oppen completeness theorem simply because solvers exchange equalities. Finite datatypes and fixed-width bitvectors do not satisfy the usual stably-infinite assumptions, and dependent encodings add further issues. The proof-producing architecture supports sound exchanges without claiming a blanket completeness result for the combined theory.

10 Trust profiles and final acceptance

10.1 Kernel checking and native evaluation are different profiles

The current Lean tactic reference explicitly distinguishes kernel reduction from native evaluation: `decide +native` and `native_decide` enlarge the trusted base, and the `bv_decide` documentation notes code-generator trust and generated axioms. Merely caching an LRAT file for `bv_check` does not, by itself, establish a kernel-only replay profile.²²

²²Lean language reference, decision procedures and SAT integration [25].

Forge should therefore have two explicit profiles. The default *standard-axiom, kernel-replay profile* permits the project’s declared logical axioms, but no newly admitted solver or native-evaluation assertions. An optional *native-evaluation profile* can accept a broader, explicitly reported dependency policy. These labels concern logical trust, not whether untrusted search is implemented in native code: an external optimizer is perfectly compatible with strict replay when its returned data is checked without trusting the optimizer.

For a certified computation, prove the checker’s soundness theorem once, then establish the concrete check with a supported kernel reduction path, such as `decide +kernel`, or with an ordinary proof term. The SOS source’s proof-facing design is useful here. For a SAT checker, an adapter still has to verify that its selected replay route meets the requested profile; it should not claim a strict route exists merely because a tactic has “check” in its name.

10.2 An explicit acceptance procedure

An eventual *forge* invocation should perform the following final checks:

1. Restore the recorded original goal context and instantiate the selected proof’s metavariables. Reject any unresolved term or universe metavariables.
2. Infer and check the proof’s type against the exact original target, using only legitimate definitional equality or recorded proof transports.
3. Inspect the transitive declaration dependencies. Reject `sorryAx`, temporary admitted hypotheses, unapproved native assertions, and any axiom outside the declared policy. Existing project axioms must be reported, not quietly relabeled as part of the kernel.
4. Validate the assembled declaration through Lean’s normal checking path. For reproducibility, replay the exported proof in a clean pinned workspace with the original pre-theorem environment.

Type inference in a running metaprogram is useful but is not the whole audit. An expression may type-check because an imported axiom states the desired result. Likewise, a tactic can appear to close the metavariable while its suggested source script leaves a hint unproved. Both require scrutiny beyond the visible number of remaining tactic goals.

10.3 Python certificates are not Lean proofs

The Python checker suite shares low-level term and rational-polynomial code with the search programs, though its replay algorithms do not call the search procedures. Differential tests against SymPy reduce the risk of some arithmetic implementation bugs, but they do not formally verify Python, its runtime, or the data model. The correct claim is *independently replayed exact certificates for a specified fragment*, not kernel-checked Lean theorems.

The command `python -S scripts/verify_certificates.py` is useful evidence of separation: it replays persisted certificates without importing SciPy, NumPy, or SymPy. It is not a replacement for porting the checker and its soundness proof into Lean. The JSON parser also is not a hardened untrusted network service; production deployments need size, nesting, time, and memory limits in addition to mathematical validation.

11 Scheduling, retrieval, and proof-size control

11.1 A deterministic initial schedule

Start with a deterministic policy that can be reproduced and ablated. Give cheap normalization and the pinned baseline a short initial opportunity. Thereafter maintain separate queues for local saturation, theorem demands, structural plans, witness candidates, nonlinear certificates, and external logical search. Use fixed quanta plus a small fair-widening quota. Exact numeric weights should be tuning parameters, not hard-coded claims about all goals.

One reasonable initial policy ranks actions by

$$\frac{\text{estimated probability of closing an obligation} \times (1 + \text{waiting consumers})}{\text{estimated search cost} + \text{estimated replay cost} + 1},$$

subject to periodic service for each enabled queue. Initial estimates can be simple tables by goal class; no trained model is required. A learned ranking model may later improve efficiency, but it remains an untrusted heuristic and cannot modify the proof acceptance rules.

Record a budget vector, not just a timeout: Lean heartbeats, external wall time, maximum term generation, theorem instances, induction depth, candidate grammar size, polynomial degree, certificate term count, coefficient bit length, and replay steps. Every expensive engine should return an explicit exhaustion reason. A mathematical failure and a resource failure must not have the same diagnostic.

11.2 Library retrieval driven by missing premises

The baseline already has a suggestion mechanism. Forge should improve how retrieval is used, not claim to introduce library search. Query a theorem index with both the goal and currently blocked premises. Prefer type-compatible conclusion heads, operations appearing in the residual, and theorems whose premises have likely producers in the active engines.

Use staged top- k retrieval: a small set first, then larger sets if search stalls. In a strict benchmark, build the index only from the original pre-theorem environment. Never inspect the target theorem's existing proof to choose premises unless the experiment is explicitly labeled an oracle-premise upper bound. Attributes that indirectly reintroduce a held-out theorem are also leakage.

Retrieval can propose an induction lemma, a bridge theorem, or a normalization identity. It need not add every retrieved theorem to every solver. Record a small premise slice for each engine call, then minimize that slice after a successful proof if doing so makes replay materially cheaper.

11.3 Avoiding repeated work and invalid caches

A cache key must include the environment identity, original typed expressions, local-context identity, universe instantiations, transparency policy where relevant, and the dependency support of branch assumptions. Alpha-normalized pretty-printed goals are not enough. A proof of $P(x)$ under $h : x = 0$ is not a proof of the same displayed goal under $h : x = 1$.

For an initial implementation, restrict cross-branch sharing to closed theorems or explicitly abstracted proofs. Share more aggressively only after context transport is implemented and tested. Persist certificate data and source-level replay recipes, not arbitrary serialized mutable metavariable contexts.

Proof construction also has a budget. Hash-cons repeated certificate subterms, share derived lemmas, and prefer a compact algebraic identity over thousands of small rewrites when its checking cost is lower. Preserve a readable proof script where practical, but retain the exact certificate when a short script would rerun nondeterministic search.

12 Concrete Lean implementation plan

12.1 Integration points that actually exist

The pinned source contains `Lean.Meta.Grind.main`, parameter constructors including `mkDefaultParams`, and a result structure with failure and instrumentation information. There is an extra-fact path that consumes proofs. These give a concrete starting point for an adapter.²³

The internal state includes proof-bearing new facts, equality information, generations, and saved metaprogramming state. It is substantial reusable infrastructure, but it is not evidence of a stable public plugin interface for all of Forge’s proposed operations.²⁴

The safest first version operates at coarse granularity: construct local Lean subgoals, run selected engines on those goals, collect closed proof terms, and restart or re-enter `grind` with the proved consequences. This costs more than a fully incremental integration but avoids unsafely mutating undocumented internal structures. A later version can add narrow upstream hooks for certified fact insertion, demand notification, resumption, and diagnostics.

12.2 Proposed module layout

Module	Responsibility and first implementation choice
<code>Forge.Core</code>	Session identities, typed facts, obligations, dependencies, budgets, and event vocabulary.
<code>Forge.Check</code>	Exact-target checking, axiom policy, certificate replay registration, and final export.
<code>Forge.Search</code>	AND/OR scheduler and transactional branch snapshots; borrow Aesop design patterns or integrate suitable internals behind an adapter.
<code>Forge.Demand</code>	Conclusion index, typed backward matching, premise joins, and demand/forward coordination.
<code>Forge.Induction</code>	Recursive-position analysis, dependency-aware generalization, recursor application, and residual extraction.
<code>Forge.Lemma</code>	Typed grammar enumeration, safe test filters, anti-unification proposals, and the checked theorem bank.

²³Lean 4.34.0, `Lean/Meta/Tactic/Grind/Main.lean` [26].

²⁴Lean 4.34.0, `Lean/Meta/Tactic/Grind/Types.lean` [27].

Module	Responsibility and first implementation choice
Forge.Witness	Typed candidate terms and generation of exact witness obligations.
Forge.Nonlinear	Adapters for cheap arithmetic, compatible SOS engine, and the polynomial/box certificate routes.
Forge.Bridges	Proved representation conversions and their guards; no global semantic erasure.
Forge.Backends	Versioned Grind, Duper, Lean-SMT/QuerySMT, and optional SAT interfaces.
Forge.Replay	Stable proof scripts/certificate files, dependency manifests, and deterministic replay without numerical search.

The separation between Core, Check, and external-search modules allows replay-only builds. A numerical solver can be absent on the machine checking a finished proof. Compatibility wrappers belong in their own modules; a Lean upgrade should not require editing every search algorithm.

12.3 A proposed user interface

The following syntax expresses intended behavior only. It is not accepted by the supplied files, which do not define a `forge` elaborator.

```
by
  forge

by
  forge (config := {
    induction := true
    synthesizeLemmas := true
    nonlinear := true
    externalSearch := false
    trust := .standardAxioms
  }) [helper_theorem]

by
  forge? -- suggest a replayable script/certificate invocation
```

A failure report should name the smallest remaining obligations and the exhausted budgets. For example: “Induction step requires a lemma about the changing accumulator; 128 candidate expressions rejected by tests; 3 universal proofs remain open.” Another useful report is: “SOS candidate rejected by exact coefficient check; no theorem added.” Such messages help distinguish a missing library theorem, a poor search bound, and a genuine mathematical issue.

12.4 Implementation stages with acceptance gates

Stage A: trustworthy orchestration. Implement the state model, transactional goal isolation, coarse grind adapter, fact support, exact-goal validation, and replay export. Acceptance requires regression tests for sibling-branch leakage, metavariable contamination, incorrect transports, and denied axioms. A successful grind proof should survive orchestration without logical changes.

Stage B: the first major capability extension. Port the list-fragment induction checker or generate ordinary Lean induction proofs, then implement recursive-position motive selection and changing-parameter generalization. Add the bounded lemma grammar. Acceptance requires solving unseen variations of accumulator correctness with the correctness theorem absent from the environment, together with false-statement and circular-lemma rejection tests.

Stage C: theory and witness integration. Align a specific SOS/Hex/Mathlib manifest with the target Lean version; expose certified nonlinear consequences as reusable facts; connect witness enumeration and guard demands. Add a Lean Bernstein soundness theorem and exact covering-tree replay if benchmarks justify it. Acceptance requires certificates replaying without a numerical solver and mixed examples needing both structural and arithmetic reasoning.

Stage D: wider quantified and external search. Add the demand index, semi-naive premise joins, and optional reconstruction-capable external engines. Add a strict SAT path only when its checker and trust policy are established. Acceptance requires zero unproved hints in all counted successes, explicit compatibility manifests, and comparisons against the strongest configured baseline rather than default grind alone.

Stage E: optimize from measurements. Introduce persistent incremental hooks, learned scheduling, stronger lemma synthesis, and richer algebraic certificates only after profiling demonstrates their value. Measure replay cost as well as search cost. These are deliverable stages rather than promised calendar estimates; the difficult work lies in dependent-context handling, version alignment, and proof reconstruction, not in writing the tactic keyword.

13 Executed experiments and reproducibility

13.1 Environment and measured scope

The recorded run uses Python 3.13.5 on Linux x86-64, NumPy 2.3.5, SciPy 1.17.0, SymPy 1.14.0, and pytest 9.0.2. The exact replay modules use the standard library; NumPy/SciPy are used by numerical search, and SymPy is used for differential arithmetic tests. The saved environment and run output are in `results/results.json` and `results/experiment_log.txt`.

The suite completed with **69 passing tests**. Thirteen persisted certificate bundles then replayed successfully in a separate command run with Python’s site initialization disabled. One bundle contains an ordered bank of five inductively proved lemmas and a specialization proof; four contain Horn proofs; six contain cone certificates; one contains a box certificate; and one contains the two certificates for an existential witness. These are bundles, not 13 independent Lean theorems or 69 benchmark problems.

Raw single-run timings are retained for reproducibility, but no speedup ratio is used as an article conclusion. The examples are tiny, numerical libraries have initialization effects, and different search policies perform different amounts of work. Node counts and checked certificates are the more interpretable observations here.

Table 3: Executed mechanism results. “Checked” means the Python fragment checker.

Mechanism	Recorded result
Accumulator lemma discovery	Direct induction fails in the prototype; candidate 41 yields the generalized specification; induction and specialization check.
Additional list obligations	Five total lemmas receive checked induction certificates in dependency order.
Quartic nonnegativity	280 LP columns, 35 equations, three exact weighted-square terms.
Guarded nonnegativity	Four additional targets certified using their explicit guards.
Incomplete dictionary control	Search fails on $(x+y+1)^2$; a separately supplied one-square certificate checks.
Bernstein box reasoning	Initial bound inconclusive; one split yields two positive leaves and a three-node certificate.
Polynomial witness	Candidate 23 gives x^2+1 ; both universal inequalities check.

Table 4: Horn policy comparison; neither column is an implementation of grind.

Depth	Forward facts	Backward proof nodes	Forward cap	Forward status
4	16	5	5,000	Certified
8	256	9	5,000	Certified
12	4,096	13	5,000	Certified
16	5,000	17	5,000	Cap reached

13.2 Negative tests are part of the evidence

The tests reject negative cone weights, absent guard indices, wrong polynomial dimensions, and a target changed by only 10^{-12} . They reject an out-of-range box split, an incorrect leaf bound, a missing child, and a strict claim based on a zero lower bound. A false polynomial target produces no certificate.

Horn mutation tests reject cyclic parent indices, the wrong rule, the wrong final goal, and a removed premise. A guarded rule cannot fire when its guard is missing. The backward engine’s unsupported unbound-premise case is tested to abstain. Induction tests reject a base-case use of the induction hypothesis, invalid generalization metadata, external rigid-variable intrusion, a false equality, and a reference to an unavailable future lemma. Removing required accumulator generalization breaks the certificate as expected.

Twelve seeded tests compare sparse arithmetic, evaluation, and substitution against SymPy, and twelve seeded planted-cone tests check numerical search followed by exact replay. These are useful robustness checks, not an exhaustive proof of the implementation. The shared polynomial primitives remain part of the Python experiment’s trusted implementation base.

13.3 Reproduction commands

From the extracted archive root:

```
python -m pip install -r requirements.txt
python -m pytest -q
```

```
python scripts/run_experiments.py
python -S scripts/verify_certificates.py
python scripts/check_lean.py
```

The last command records `NOT_RUN` when Lean is absent; it must not be interpreted as a successful Lean test. When a suitable Lean executable is available, it attempts to compile the core structural reference file and records the actual version and output. The separate Mathlib examples need a compatible Mathlib project and are not compiled by that script. The article can be rebuilt with:

```
cd article
xelatex -interaction=nonstopmode -halt-on-error forge.tex
xelatex -interaction=nonstopmode -halt-on-error forge.tex
xelatex -interaction=nonstopmode -halt-on-error forge.tex
```

No external bibliography processor is necessary. Font programs are not included; the document uses fonts available in common TeX/font installations. The source and code are UTF-8.

14 A fair end-to-end evaluation of “significantly more capable”

14.1 Why the environment and completion metric matter

The QuerySMT study explicitly discusses both a success metric that can leave individual hints unproved and contamination from benchmark environments that already contain target theorems accessible through attributes. Those results are valuable but cannot be copied directly as fully verified current-version success rates.²⁵

Forge should count a solved problem only when the exported proof checks in the pre-theorem environment and passes the declared axiom policy. Adding a theorem alias in a file that imports the original theorem is not a sufficient holdout strategy. The test harness should construct the declaration environment before the target is introduced, or reproduce a source prefix with an appropriately isolated tactic import layer.

14.2 Benchmark families and baselines

Use separate tracks for ordinary mathematical developments and deliberately minimal libraries. The first measures practical utility with normal imports; the second measures structural and lemma-synthesis capability when the exact answer is absent. Include recursive list/tree/accumulator correctness, quantified equality and Horn chains, non-linear inequalities, existential witnesses, and mixed structural/arithmetic obligations. Preserve false and ill-typed near-neighbors as soundness regressions, not as ordinary success-rate items.

Compare at least: pinned default `grind`; `grind` with suggestions and reasonably tuned budgets; a simple Aesop/Grind portfolio; compatible Duper and reconstruction-based SMT routes; and a basic portfolio with the existing SOS tactic. Otherwise a gain attributable merely to adding a known solver may be mistaken for a gain from Forge’s coordination.

Use the same pre-theorem declarations, permissible retrieved premises, total wall-clock budget, memory limit, and trust profile. External solver time, translation, reconstruction,

²⁵QuerySMT methodology and evaluation discussion, Sections 8.1–8.2 [11].

proof export, and replay must all be charged. Report cold-start and warm-cache conditions separately. Tune on development theorems; hold out entire modules or definition families for testing so near-duplicate lemmas do not dominate the results.

14.3 Metrics, ablations, and a concrete acceptance target

Report solved counts by family, uniquely solved cases, failures by reason, median and tail latency, proof bytes, certificate bytes, reconstruction time, and checking time. A conditional proof with one open guard belongs in a partial-progress category. A model of an abstraction belongs in a diagnostic category until it is validated against the original statement.

Ablate induction planning, lemma synthesis, demand propagation, witness synthesis, nonlinear fact exchange, and proof-cost-aware scheduling separately. Most importantly, compare the full planner against the *same engines run without feedback*. That identifies whether the architecture contributes more than a portfolio union.

A useful engineering gate would require a predeclared improvement on at least two structurally different held-out families, together with a bounded regression rate on easy baseline successes and no trust-policy violations. The numerical thresholds should be set before benchmarking, for example a target of 20% more fully checked solves on the selected difficult families and at most 5% regression on the baseline suite. These numbers are proposed acceptance criteria, *not achieved results or forecasts*. No such Lean evaluation has been run for this artifact.

15 Limits, priorities, and conclusion

The largest practical risk is search explosion. Induction choices multiply subgoals; lemma grammars grow exponentially; higher-degree polynomial certificates become large; backward quantifier search can branch as badly as forward saturation. A planner makes these choices explicit and budgeted, but does not remove their underlying difficulty. Completeness claims must always refer to a specified fragment and a specified unbounded search policy.

The second risk is integration rather than mathematics: mutable metavariable state, dependency transport, representation semantics, version mismatch, and large proof terms. The staged design prioritizes these boundaries before advanced heuristics. A restricted correct implementation that exports a small set of ordinary Lean proofs is preferable to a wide solver bridge that silently leaves obligations or enlarges the trust base.

The most valuable first release is therefore not “every solver behind one keyword.” It is a planner that automatically discovers and proves useful induction strengthenings, synthesizes witnesses whose obligations can be certified, and coordinates those results with the already strong Grind, SOS, and logical engines. The Python experiments show that these mechanisms can be made concrete, inspectable, and certificate-driven. They do not establish that a finished Forge tactic outperforms current Lean automation.

Bottom line. There is a technically credible route to substantially broader automation than a local grind call: let the tactic search for the missing proof structure, not only for more consequences of a fixed local state. The next decisive milestone is a Lean implementation of the scoped planner and structural synthesis path, followed by clean-environment, fully-checked comparisons against a strong multi-engine baseline.

A Artifact map and status ledger

Path	Contents and validation status
article/forge.tex, forge.pdf forge_lab/terms.py	This article and its rendered PDF. Typed first-order terms, fixed list equations, rewrite search and independent replay. Executed.
forge_lab/induction.py	Generalization-specific lemma enumeration and list induction certificates. Executed.
forge_lab/horn.py	Two bounded Horn search policies and a rule-instance DAG checker. Executed.
forge_lab/poly.py	Exact sparse rational arithmetic, cone checker, Bernstein transformation, and covering-tree checker. Executed.
forge_lab/nonlinear.py	Untrusted SciPy LP search and bounded exact box search. Executed.
forge_lab/witness.py	Polynomial witness enumeration with two universal certificates. Executed.
tests/test_certificates.py	69 tests, including arithmetic differential tests and deliberate certificate corruptions. Passed.
results/	Raw outputs, JSON certificates, count CSV, test/replay logs, and explicit Lean nonexecution status.
scripts/	Experiment, standard-library replay, and optional Lean compilation commands.
lean/Structural.lean	Ordinary core Lean proofs corresponding to the structural examples. Supplied, not compiled.
lean/CertificateExamples.lean	Mathlib proof scripts illustrating the arithmetic certificates. Supplied, not compiled.
sources.json	Bibliographic and source-version inventory with original URLs.

B Certificate schemas and their interpretation

A cone record contains the exact target polynomial, an ordered list of guards, and terms (c, q, J) . The interpretation is $p = \sum c q^2 \prod_{j \in J} g_j$, conditional on each listed guard being nonnegative. All rational values are serialized as strings, never floating-point JSON numbers. Variable count and monomial exponent vectors are explicit.

A box record contains the exact polynomial, the root box, a strictness flag, and a tree of leaves or interior splits. The checker reconstructs child boxes; there is no trusted user-supplied coverage assertion. A witness record contains one univariate polynomial and cone certificates for its difference from x and from $-x$.

A Horn record contains the original premises, named rules, exact target, and proof DAG. Its validity is conditional on the listed premises and rules; those inputs are the problem statement, not new discoveries. A list-induction record contains the candidate equality,

major variable, generalized variables, and four rewrite traces: left/right traces for the base and step cases. The theorem bank's order supplies the allowed previously checked lemmas.

The saved records are intended to be portable explanatory artifacts for the prototype. They are not yet an agreed wire format for Lean. A production format must additionally bind every object to a versioned typed interpretation and context manifest, impose resource limits, and have a checked Lean decoder or an equally well-justified elaboration path.

C A compact replay target for the discovered lemma

The following intended Lean proof explains the desired final output of structural synthesis. The full reference file defines the recursive operations and preceding concatenation lemmas. This source was not compiled here.

```
theorem revAcc_spec (xs acc : List  $\alpha$ ) :
  revAcc xs acc = app (rev xs) acc := by
  induction xs generalizing acc with
  | nil => rfl
  | cons x xs ih =>
    simp only [revAcc, rev, ih, app_assoc, app]

theorem revAcc_correct (xs : List  $\alpha$ ) :
  revAcc xs [] = rev xs := by
  rw [revAcc_spec, app_right_nil]
```

The final theorem does not invoke the synthesizer. It reuses a separately proved general lemma. This is the preferred deployment pattern: potentially expensive search once, then a stable and understandable proof for ordinary recompilation.

Sources

- [1] Lean project. *The grind tactic*. Lean Language Reference. Observed September 14, 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [2] Lean project. *Lean 4.34.0 release*. September 14, 2026. <https://github.com/leanprover/lean4/releases/tag/v4.34.0>.
- [3] Lean project. *Grind configuration*, tag v4.34.0. <https://raw.githubusercontent.com/leanprover/lean4/v4.34.0/src/Init/Grind/Config.lean>.
- [4] Lean project. *Grind commutative-ring constraint types*, tag v4.34.0. <https://raw.githubusercontent.com/leanprover/lean4/v4.34.0/src/Lean/Meta/Tactic/Grind/Arith/CommRing/Types.lean>.
- [5] Lean project. *Linear Integer Arithmetic*. Lean Language Reference. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/Linear-Integer-Arithmetic/>.
- [6] Lean community. *Aesop: White-box automation for Lean 4*. Repository documentation. <https://github.com/leanprover-community/aesop>.
- [7] Lean community. *Duper*. Repository documentation. <https://github.com/leanprover-community/duper>.
- [8] Lean community. *Lean-auto: Experiments on automation for Lean*. Repository documentation. <https://github.com/leanprover-community/lean-auto>.
- [9] Yicheng Qian, Joshua Clune, Clark Barrett, and Jeremy Avigad. *Lean-auto: An Interface between Lean 4 and Automated Theorem Provers*. 2025. <https://arxiv.org/abs/2505.14929>.
- [10] Lean-SMT project. *Lean-SMT*. Repository documentation. <https://github.com/ufmg-smite/lean-smt>.
- [11] Joshua Clune, Haniel Barbosa, and Jeremy Avigad. *Hint-Based SMT Proof Reconstruction*. January 2026, arXiv:2601.14495v1. <https://arxiv.org/html/2601.14495v1>.
- [12] Lean SOS project. *Harrison's sum-of-squares decision procedure for nonlinear real arithmetic in Lean 4*. Repository documentation. <https://github.com/leanprover/sos>.
- [13] Lean SOS project. *SOS/Engine.lean*, observed main source, September 14, 2026. <https://raw.githubusercontent.com/leanprover/sos/main/SOS/Engine.lean>.
- [14] Lean SOS project. *SOS/Core.lean*, observed main source. <https://raw.githubusercontent.com/leanprover/sos/main/SOS/Core.lean>.
- [15] Lean SOS project. *lean-toolchain*, retrieved main snapshot specifying v4.32.0-rc1; not a verified common build manifest. <https://raw.githubusercontent.com/leanprover/sos/main/lean-toolchain>.
- [16] Lean Hex project. *hex-mv-poly: Mathlib-free executable multivariate polynomials for Lean 4*. <https://github.com/leanprover/hex-mv-poly>.
- [17] Mathlib community. *Mathlib.Tactic.Linarith.Frontend*. API documentation. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Linarith/Frontend.html.
- [18] Mathlib community. *Mathlib.Tactic.Positivity.Basic*. API documentation. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Positivity/Basic.html.
- [19] Marcus Rossel and contributors. *lean-egg*. Repository, including current deprecation notice. <https://github.com/marcusrossel/lean-egg>.
- [20] E-graphs community. *egg: e-graphs good*. Rust library repository. <https://github.com/egraphs-good/egg>.

- [21] Yican Sun, Ruyi Ji, Jian Fang, Xuanlin Jiang, Mingshuai Chen, and Yingfei Xiong. *Proving Functional Program Equivalence via Directed Lemma Synthesis*. 2024. <https://arxiv.org/abs/2405.11535>.
- [22] SciPy developers. *linprog(method='highs')*. Optimization API documentation. The executed artifact uses SciPy 1.17.0; the moving online documentation may differ. <https://docs.scipy.org/doc/scipy/reference/optimize.linprog-highs.html>.
- [23] COIN-OR. *CSDP: A solver for semidefinite programming problems*. <https://github.com/coin-or/Csdp>.
- [24] SymPy developers. *Polynomial Manipulation: Reference*. <https://docs.sympy.org/latest/modules/polys/reference.html>.
- [25] Lean project. *Tactic Reference*, especially decision procedures, native evaluation, and SAT integration. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>.
- [26] Lean project. *Grind main entry points*, tag v4.34.0. <https://raw.githubusercontent.com/leanprover/lean4/v4.34.0/src/Lean/Meta/Tactic/Grind/Main.lean>.
- [27] Lean project. *Grind state and proof-bearing types*, tag v4.34.0. <https://raw.githubusercontent.com/leanprover/lean4/v4.34.0/src/Lean/Meta/Tactic/Grind/Types.lean>.